Aufgabe 1: Anfrage bearbeiten (Pflicht, 30 Minuten)

Gruppe: _____ Instanz: gtm ___

Ziel: Ein Formular nimmt Anfragen an. Dein Workflow findet die passende Ansprechperson und antwortet je nach Dringlichkeit.

Dein Start: In deiner n8n-Instanz öffnest du den Workflow *Aufgabe 1: Anfrage bearbeiten (Start)*. Darin liegt schon der Formular-Trigger.

Schritte

1. **Formular testen.** Doppelklick auf den Trigger, *Execute step*, Formular ausfüllen und abschicken. Tipp: Danach den Trigger **anpinnen** (Pin-Symbol), dann musst du nicht jedes Mal neu ausfüllen.
2. **HTTP Request** anhängen.
   - Methode `GET`, URL `http://tools:5678/webhook/mitarbeiter`
   - Query-Parameter `abteilung` mit dem Wert `{{ $json.Abteilung }}` (Abteilung aus dem Formular)
   - Ausführen: Du siehst Mitarbeitende dieser Abteilung.
3. **IF** anhängen: Ist die Dringlichkeit gleich `hoch`?
   Achtung: Nach dem HTTP Request ist `$json` die Antwort der API. Das Formular erreichst du mit `{{ $('Anfrage-Formular').item.json.Dringlichkeit }}`.
4. **Zwei Edit Fields** (einer pro Zweig). Jeder baut ein Feld `antwort`, z. B.
   - dringend: `Danke {{ $('Anfrage-Formular').item.json.Name }}, wir melden uns heute noch. Ansprechperson: {{ $json.mitarbeiter[0].name }}`
   - normal: `Danke, wir melden uns in den nächsten Tagen.`
5. **Form** (Operation *Form Ending*) an beide Zweige hängen.
   - *Completion Title*: `Anfrage eingegangen`
   - *Completion Message*: `{{ $json.antwort }}` (auf **Expression** umschalten)

Fertig, wenn

Du das Formular mit *hoch* und mit *normal* abschickst und jedes Mal die passende Antwort siehst.

Bonus (wenn du früh fertig bist)

Baue eine **Basic LLM Chain** ein, die die Nachricht in einem Satz zusammenfasst. Modell: **OpenRouter Chat Model**, Credential *OpenRouter (Workshop)*, Modell z. B. `openai/gpt-4o-mini`.

Zum Nachdenken

Die Mitarbeiter-Daten kommen über eine **REST-Schnittstelle** (HTTP GET mit Parametern). Jedes Tool mit so einer Schnittstelle kannst du genauso anbinden.

Aufgabe 2: Ein Agent mit Tools (erwünscht, 25 Minuten)

Gruppe: _____ Instanz: gtm ___

Ziel: Ein **AI Agent** entscheidet selbst, welche Werkzeuge er benutzt. Alle Bausteine liegen schon da, du verbindest sie.

Dein Start: Workflow *Aufgabe 2: Agent (Start)*.

Schritte

1. **Memory verbinden.** Am AI Agent beim Anschluss *Memory* auf das **Plus-Zeichen** klicken und *Window Buffer Memory* wählen (oder vom Plus-Zeichen zum vorhandenen Node ziehen).
2. **Tool verbinden.** Am Anschluss *Tool* auf das **Plus-Zeichen**, dann *Mitarbeiter suchen*. Er ruft den Workflow *Tool: Mitarbeiter suchen* auf.
3. **Testen.** *Open chat*, dann: *Wer arbeitet im IT-Support?*
4. **Zweites Tool.** Verbinde *Abteilungen auflisten* (ruft den Workflow *Tool: Abteilungen auflisten* auf). Der Agent kann nun die genauen Abteilungsnamen nachschlagen. Teste: *Wer arbeitet in der IT?* Schau in der Ausführung, in welcher Reihenfolge er die Tools aufruft.
5. **MCP-Tool verbinden.** Verbinde *MCP Client Tools-Server*. Der Agent kennt dann auch freie Räume und das Wetter.
   Frage: *Gibt es einen freien Raum für 8 Personen?*
6. **Systemprompt schreiben.** AI Agent öffnen, *Add Option*, *System Message*. Gib Rolle, Sprache und Regeln vor, z. B. „Du bist der freundliche Assistent der Firma. Antworte kurz auf Deutsch. Erfinde keine Personen.“

Beobachte

Öffne nach einer Frage die Ausführung. Du siehst, **welches Tool** der Agent mit **welchen Eingaben** aufgerufen hat.

Fertig, wenn

Der Agent diese drei Fragen richtig beantwortet:

- Wer arbeitet im IT-Support?
- Gibt es einen freien Raum für 8 Personen?
- Wie ist das Wetter in München?

Fragen zum Ausprobieren

- Was passiert, wenn du nach jemandem fragst, den es nicht gibt?
- Wie ändert sich die Antwort mit einem anderen Systemprompt?
- Merkt sich der Agent die vorige Frage („Und wer ist dort der Leiter?“)?

Aufgabe 3 (optional, für Schnelle)

Gruppe: _____ Instanz: gtm ___

Such dir **eine** Variante aus.

A) Eigenes Tool: Räume finden

Baue einen **Tool-Workflow** (wie *Tool: Mitarbeiter suchen*), der `http://tools:5678/webhook/raeume` mit dem Parameter `plaetze` abfragt. Binde ihn an den Agenten an und lass ihn „den kleinsten freien Raum für 5 Personen“ nennen.

B) Dein Workflow als Schnittstelle

Ersetze den Formular-Trigger aus Aufgabe 1 durch einen **Webhook**. Rufe ihn mit dem Browser oder `curl` auf und gib die Antwort als JSON zurück (*Respond to Webhook*).

C) Der Agent schreibt die Antwort

Nutze in Aufgabe 1 statt der festen Antworttexte eine **Basic LLM Chain**, die je nach Dringlichkeit einen passenden Text formuliert.

Hilfekarte: die fünf häufigsten Stolperstellen

1. Ausdrücke (Expressions)

Werte aus dem vorigen Node holst du mit `{{ $json.feld }}` oder, für frühere Nodes, `{{ $('Node-Name').item.json.feld }}`. Das Feld muss genau so heißen wie in der Ausgabe (auch Groß- und Kleinschreibung: `Name` ist nicht `name`). Am einfachsten: das Feld aus dem Eingabe-Panel links in das Eingabefeld ziehen.

2. Pinning

Drück auf das Pin-Symbol in der Ausgabe eines Nodes, dann bleiben seine Testdaten erhalten. Du musst das Formular nicht neu ausfüllen. Pins entfernen: noch einmal drücken.

3. Credential auswählen

Bei Chat-Modellen im Node *Credential to connect with* **OpenRouter (Workshop)** auswählen. Einen eigenen API-Schlüssel brauchst du nicht. Modellnamen im Format `openai/gpt-4o-mini`.

4. Nodes verbinden

Ein Node ohne Linie zum nächsten läuft nicht mit. Am Agenten hängen Memory, Tools und Modell an den **Anschlüssen unten** (jeweils mit **Plus-Zeichen**), nicht an den Pfeilen rechts.

5. Test oder Produktion

Execute workflow ist ein Test mit Ansicht der Daten. Die **Production-URL** eines Formulars oder Webhooks funktioniert nur, wenn der Workflow **aktiv** ist (Schalter oben rechts, *Publish*). Test-URL und Production-URL sind verschieden.

Hilft alles nichts?

- Oben rechts *Executions*: Dort steht jede Ausführung mit Fehlermeldung am roten Node.
- Rote Fehlermeldung vorlesen. Meistens steht der Grund im ersten Satz.
- Frag dein Team, dann Rudi.

Adressen im Workshop

- Mitarbeiter-API: `http://tools:5678/webhook/mitarbeiter` (Parameter `name`, `abteilung`)
- Räume-API: `http://tools:5678/webhook/raeume` (Parameter `plaetze`)
- Abteilungs-API: `http://tools:5678/webhook/abteilungen` (keine Parameter)
- MCP-Server: `http://tools:5678/mcp/workshop`